Veri Yapıları

Yığınlar

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

Yığın (Stack) ADT

- Yığın, **veri ekleme (insert)** ve **veri silme (delete)** işlemlerinin yalnızca listenin **tepe (top)** adı verilen tek bir kısıtlı ucundan yapılabildiği **özel bir liste türüdür.**
- Yığınlar yapıları gereği **LIFO (Last In, First Out - Son Giren İlk Çıkar)** prensibine sıkı sıkıya bağlı olarak çalışırlar.
- Günlük hayatta yaylı bir mekanizmaya sahip tabaklıklar veya üst üste dizilmiş kitaplar bu veri yapısını anlamak için en iyi fiziksel örneklerdir.

Yığın Modelinin Temel İşlemleri

- Bir yığın modelinde veriler üzerinde serbestçe dolaşmaya izin verilmez; **yalnızca en üstteki veri görünürdür.**
- **Temel işlemler şunlardır:**
  - **Push:** Listeye yeni bir elemanı en üstten (tepe noktasından) girdi (input) işlemiyle ekler.
  - **Pop:** Listeye en son eklenen elemanı siler ve bir çıktı (output) işlemi olarak listeden çıkarır.
  - **Top:** Yığının en tepesinde bulunan, yani en son eklenen elemanı silmeden yalnızca değerini okumak için kullanılır.

ADT Kuralları ve Hata Yönetimi (Error Handling)

- Veri yapılarında kısıtlamalar, hataları da beraberinde getirir.
- Boş bir yığın (empty stack) üzerinde **pop** veya **top** işlemleri yapmaya çalışmak, yığın ADT'si kuralları gereği genel bir hata (error) olarak kabul edilir.
- Öte yandan, çok fazla eleman ekleyip sistemin hafızasını doldurarak bir **push** işleminde hata almak (out of space), yığın ADT'sinin kurallarıyla ilgili bir hata değildir; bu tamamen programın bellek limitleriyle (implementation limit) ilgilidir.

Zaman Karmaşıklığı Analizi (Time Complexity)

- Yığın operasyonları, doğru kodlandığı takdirde döngülere veya verilerin kaydırılmasına ihtiyaç duymaz.
  - Bu nedenle **push**, **pop** ve **top** operasyonlarının **her biri O(1)** (sabit zamanlı) operasyonlardır.
- Yığınlar donanım seviyesinde de desteklenir.
- Birçok modern bilgisayar mimarisinde, **push** ve **pop** işlemleri, **işlemcinin komut seti (instruction set)** düzeyinde **otomatik artırma/azaltma (auto-increment/auto-decrement)** yapan kaydedicilerle tek bir makine komutu hızında dahi çalıştırılabilmektedir.

Yığınların Gerçeklenmesi (Implementation of Stacks)

- Yığın yapısal olarak aslında kısıtlanmış bir liste (list) olduğundan, C++ dilinde liste yapısını destekleyen herhangi bir veri yapısı ile gerçeklenebilir.
- C++ STL (Standard Template Library) içindeki **vector** ve **list** sınıfları yığın işlemleri için biçilmiş kaftandır ve programcıların büyük çoğunluğu bu hazır yapıları kullanır.
- Özel performans ve donanım gereksinimleri olduğunda, yığınlar **Bağlı Liste (Linked List)** veya **Dizi (Array)** ile sıfırdan inşa edilebilir.

Gerçekleme 1
Bağlı Liste (Linked List) Mantığı

- Yığınlar bellek israfını önlemek amacıyla **tek yönlü bağlı liste (singly linked list)** kullanılarak kodlanabilir.
- Bu modelde listeyi **baştan sona dolaşmaya gerek yoktur**. Tüm işlemler listenin **en başındaki (front) düğüm** üzerinde gerçekleştirilir.
- Bir **push** işlemi, **listenin en başına yeni bir düğüm (node)** oluşturarak ve bağlarını güncelleyerek anında yapılır.

Bağlı Listede Pop ve Top Operasyonları

- Bağlı liste gerçeklemesinde **pop** işlemi, **dizinin ilk elemanının işaretçisini koparıp (delete ile silip)** listenin başlangıcını bir sonraki elemana kaydırmak suretiyle yapılır.
- **top** işlemi ise sadece **front (baş) düğümünün** verisini döndürür.
- Bazen hız ve sadelik sağlamak için **pop** ve **top** işlemleri birleştirilerek tek bir fonksiyon (hem değeri döndüren hem silen) halinde kullanılabilir.

Gerçekleme 2
Dizi (Array) ile Yığın Mimarisi

- Bağlı listelerdeki **düğüm oluşturma (new node) ve işaretçi (pointer) bağlama maliyetinden kaçınmak için** yığınlar çok daha popüler olan **dizi**lerle de gerçeklenebilir.
- C++ **vector** sınıfındaki **back**, **push_back** ve **pop_back** fonksiyonları aslında tamamen bir yığın dizisi modelidir.
- **İşaretçi (pointer) tutmak yerine basit bir tamsayı indisi** ile takip yapılır.

Dizi Gerçeklemesinde 'topOfStack' Değişkeni

- Dizi yığınlarında iki temel veriye ihtiyaç vardır:
  - Verileri tutan **theArray** dizisi
  - Yığının güncel tepe noktasını işaret eden bir **topOfStack** tamsayısı

Kritik Kural

- Yığın tamamen boşken **topOfStack** değişkeninin değeri her zaman −1 olarak başlatılır. Bu durum yığının başlangıç/boş durumudur.

Dizi Gerçeklemesinde Hızlı Push İşlemi

- Bir **x** elemanı yığına itileceğinde (**push**):
  - Öncelikle tepe indeksini tutan **topOfStack** değişkeni bir (1) artırılır.
  - Ardından, **x** elemanı dizinin o anki yeni tepe indisine atanır.
    - **theArray[topOfStack] = x**
- Bu işlem döngü barındırmaz ve anlık çalışır. C++ dilinde bu iki işlem **theArray[++topOfStack] = x** şeklinde tek satırda yazılabilir.

Dizi Gerçeklemesinde Hızlı Pop İşlemi

- Bir eleman yığından silineceği zaman (**pop**):
  - Dizinin tepe noktasında tutulan eleman olan **theArray[topOfStack]** geçici bir değişkene alınır.
  - Daha sonra yığının boyutunu küçültmek (elemanı silinmiş saymak) için **topOfStack** değişkeni bir (1) azaltılır.
- Böylece bellekten asıl veriyi temizlemeye bile gerek kalmadan, sınır daraltılarak silme işlemi çok hızlı gerçekleştirilir.

Özet

- **Yığınlar (Stacks) bilgisayar bilimlerindeki dizilerden sonraki en temel ikinci veri yapısı olarak kabul edilir.**
- Arayüz (Interface) kısıtlıdır; verilerin arasına ekleme yapılamaz. Ancak bu kısıtlama, operasyonların hızının O(1) olmasını ve muazzam bir işletim sistemi desteği görmesini sağlamıştır.

Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

Veri Yapıları

Yığınlar
(Uygulama Dersi)

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

Yığın Uygulamaları

- Yığınlardaki kısıtlı işlem modelinin (sadece uçtan işlem), bilgisayar bilimlerinin en karmaşık sorunlarını basitçe çözebildiği algoritmik senaryoları inceleyeceğiz.
- **Odaklanacağımız uygulamalar:**
  - Derleyici tasarımı (Compiler Design)
  - Matematiksel ifadeler (Hesap Makineleri)
  - Fonksiyon/Hafıza Yönetimi

Uygulama 1
Sembol Eşleştirme (Balancing Symbols)

- Derleyicilerin (compilers) yazdığımız kodlardaki **eksik parantez** veya **kapanmamış süslü parantez** hatalarını anında bulması gereklidir.
- Sadece parantezleri **()**, köşeli ayraçları **[]** ve süslü parantezleri **{}** içeren bir yapının tamamen dengeli (balanced) olup olmadığını kontrol etmek yığınlar sayesinde oldukça kolaydır.
- Örneğin **[()]** doğru bir kod dizilimi iken, **[(])** yanlış bir dizilimdir.

Uygulama 1
Sembol Eşleştirme (Balancing Symbols)

Sembol Eşleştirme Algoritmasının Adımları

1. Boş bir yığın oluşturulur.
2. Dosya sonuna (EOF) gelene kadar karakterler okunur.
3. Eğer karakter bir açılış sembolü ise (**(**, **[**, **{**), derhal yığına itilir (**push**).
4. Eğer karakter bir **kapanış sembolü** ise, yığından bir eleman çekilir (**pop**). Çekilen açılış sembolü ile okunan kapanış sembolü eşleşmiyorsa **sözdizimi hatası (syntax error)** verilir.

Uygulama 1
Sembol Eşleştirme (Balancing Symbols)

Sembol Eşleştirmede Kritik Kontroller ve Hız

- Eğer **bir kapanış sembolü okunduğunda yığın boş ise** (karşılığı olan açılış sembolü hiç girilmemişse) doğrudan **hata verilir**.
- Eğer kod dosyasının tamamı okunup bittiğinde, **yığında hala açılış sembolleri bekliyorsa (kapatılmamış bloklar varsa) hata verilir**.
- Bu algoritma, milyonlarca satırlık kodu geriye dönüp bakmadan, tek bir geçişte (one pass) kontrol edebildiği için tam olarak **O(N)** doğrusal zamanlı çalışır ve **çok hızlıdır**.

Uygulama 2
Ters Leh (Postfix) Gösterimi

- İnsanların matematikte kullandığı **Infix** gösteriminde çarpma işlemi önceliklidir ve algoritmaların bunu çözebilmesi için parantezlere ihtiyacı vardır.
  - Örn: **4.99 + 5.99 * 1.06** gibi
- Hesap makineleri ise **Postfix (Ters Leh)** gösterimi kullanır
  - Örn: **4.99 1.06 * 5.99 +**
- **Bu formatta işlem önceliğine (precedence) ve parantezlere hiçbir şekilde ihtiyaç duyulmaz.**

Uygulama 2
Ters Leh (Postfix) Gösterimi

Postfix Algoritmasının (Hesap Makinesi) İşleyişi

- Postfix ifadeler **yığın (stack)** ile **O(N) sürede** şu adımlarla çözülür:
  - İfade soldan sağa taranır.
  - Eğer bir sayı (operand) okunursa doğrudan yığına **push** edilir.
  - Eğer bir operatör (+, * vb.) okunursa, yığından iki sayı **pop** edilir.
  - Operatör bu iki sayı üzerinde çalıştırılır ve çıkan sonuç tekrar yığına **push** edilir.
- **İfade bittiğinde yığında kalan tek eleman** matematiksel **problemin cevabıdır**.

Uygulama 2
Ters Leh (Postfix) Gösterimi

Postfix Değerlendirme Örneği (Adım Adım)

- İfade: **6 5 2 3 + 8 * + 3 + ***
- Adımlar:
  - **6, 5, 2, 3** değerleri yığına sırayla eklenir (**push**).
  - **+** okunduğunda **3** ve **2** çekilir (**pop**), toplanır (**5**) ve tekrar eklenir (**push**).
  - **8** eklenir. Stack durumu: **(Alttan üste) 6, 5, 5, 8**.
  - ***** okunduğunda **8** ve **5** çekilir, **40** olarak geri eklenir. Stack durumu: **6, 5, 40**.
  - İşlemler aynı kurallar ile devam eder ve yığında kalan son eleman sonuç (**288**) olur.

Uygulama 3
Infix İfadelerin Postfix'e Çevrilmesi

- Derleyicilerin, kullanıcıların girdiği standart infix kodları (**a + b * c**) kendi anladıkları postfix koda (**a b c * +**) çevirmesi gereklidir.
- Bu algoritmada, **sayılar/değişkenler (operandlar) okunduğu an doğrudan çıktı (output) ekranına yazdırılır**ken, **operatörler geçici olarak yığında beklemeye (pending)** alınır.
- **(sol parantezi** okunursa en yüksek önceliğe sahipmiş gibi yığına itilir, ancak **) sağ parantezi** geldiğinde, **yığındaki tüm operatörler sol parantez bulunana kadar boşaltılıp çıktıya yazılır.**

Uygulama 3
Infix İfadelerin Postfix'e Çevrilmesi

Infix'ten Postfix'e Çevrim - Öncelik Kuralları

- Okunan bir operatör (**+**, ***), yığındaki bekleyen operatörlerden daha düşük veya eşit önceliğe sahipse, önce yığındaki operatörler **pop** edilip çıktıya yazdırılır, ardından yeni okunan operatör yığına itilir.
- **Girdinin sonuna ulaşıldığında (EOF)**, yığında kalan tüm bekleyen operatörler **pop** edilip peş peşe çıktıya eklenerek çevrim **O(N)** sürede tamamlanır.
- Örnek dönüşüm için: **a + b * c + (d * e + f) * g**

Uygulama 4
Fonksiyon Çağrıları (Function Calls)

- Bilgisayar dilinde **bir fonksiyonun içerisinden başka bir fonksiyon çağrıldığında** (Örn: main içinden hesapla çağırmak), bilgisayarın **ilk fonksiyondaki yerel değişkenleri**, **yazmaç değerlerini (registers)** ve **geri dönüş adresini (return address) kaybetmemesi gerekir**.
- Bu karmaşık problem, her fonksiyon çağrısında bellekte bir **Aktivasyon Kaydı (Activation Record / Stack Frame)** oluşturulup **çağrı yığınına (call stack) push** edilmesiyle çözülür.

Uygulama 4
Fonksiyon Çağrıları (Function Calls)

Özyineleme (Recursion) ve Bellek Yapısı

- Program çağrılan yeni fonksiyondan **geri döndüğünde (return)**, yığındaki bu Aktivasyon Kaydı **pop** ile okunur ve program kaldığı adresten, mevcut değişkenleriyle eski duruma dönerek aynen devam eder.
- **Özyinelemeli (recursive) fonksiyonlarda**, fonksiyon kendi kendini **sürekli çağırdığı için** yığına art arda devasa Aktivasyon Kayıtları atılır.

Uygulama 4
Fonksiyon Çağrıları (Function Calls)

Yığın Taşması (Stack Overflow)

- Sistemler normalde yığın taşmalarına karşı kontrol yapmaz. Özyinelemeli bir fonksiyonda **temel durum (base case)** unutulursa (runaway recursion), **fonksiyon sonsuz defa çağrılır**.
- Benzer şekilde **döngü kullanılarak çözülebilecek 200.000 elemanlı bir yazdırma işlemi gereksiz yere özyineleme ile yapılırsa**, bellekte 200.000 adet büyük Aktivasyon Kaydı birikir.
- Kapasitenin tükenmesi yığın bellek kapasitesinin dolmasına ve programın **Yığın Taşması (Stack Overflow)** ölümcül hatası (fatal error) alıp aniden **çökmesine** neden olur.

Genel Özet

- Yığınlar kısıtlı arayüzleri nedeniyle yalnızca **O(1)** sabit zamanlı yüksek hız sunmakla kalmaz; derleyicilerin parantez doğrulaması, işlem önceliklerini çözümlemesi (postfix) ve en önemlisi bilgisayarın donanımsal olarak fonksiyon çağrılarını yönetebilmesinin (stack frames) arkasındaki mimaridir.
- Kod denemelerinde parantez doğrulama ve Infix/Postfix çevrim algoritmalarının **C++ üzerinde kodlaması yapılmalıdır**.

Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN